CSCE 775: Deep Reinforcement Learning
Homework #1

Instructor: Dr. Forest Agostinelli
University of South Carolina
Solutions by: JC Vaught

Spring 2026

Table of Contents

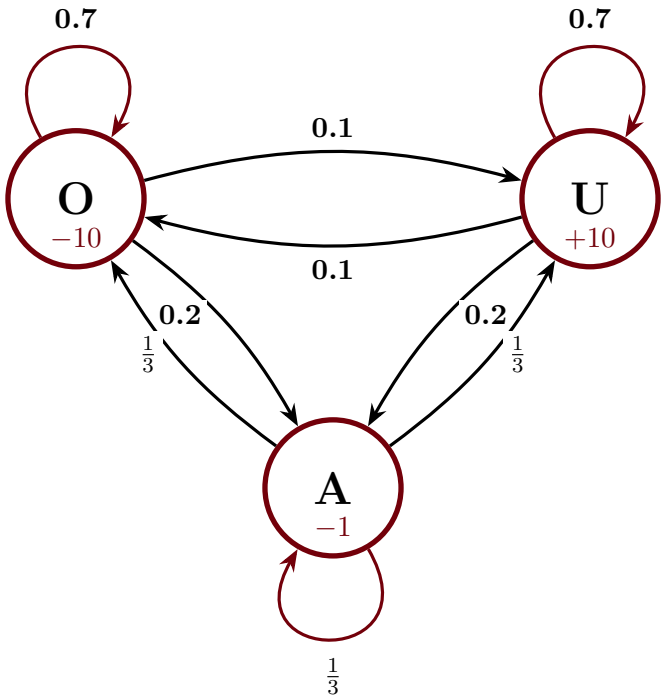
Problem 1: Markov Reward Processes (20 pts) **2**
Problem 2: Markov Decision Processes (80 pts)
2.1: Dynamic Programming (40 pts) **5**
2.2: Model-Free Reinforcement Learning (40 pts) **7**

Note: This document contains hand-worked derivations for each problem, followed by implementation sketches. The submitted code file is `code_hw/code_hw1.py`.

Problem 1: Markov Reward Processes (20 pts)

Task: Implement the `mrp` function. Given the definition of a Markov reward process, compute the state-values (expected future reward) for all states.

MRP Diagram



Problem Formulation

The state space is $\mathcal{S} = \{O, U, A\}$ with rewards $R(O) = -10$, $R(U) = +10$, $R(A) = -1$. The transition probability matrix and reward vector are:

$$\mathcal{P} = \begin{pmatrix} 0.7 & 0.1 & 0.2 \\ 0.1 & 0.7 & 0.2 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{pmatrix}, \qquad \mathbf{R} = \begin{pmatrix} -10 \\ +10 \\ -1 \end{pmatrix}$$

Solution: Direct Matrix Inversion

Step 1: Bellman Equation in Matrix Form

The value function satisfies:

$$\mathbf{V} = \mathbf{R} + \gamma \mathcal{P}\mathbf{V} \implies (\mathbf{I} - \gamma\mathcal{P})\mathbf{V} = \mathbf{R} \implies \boxed{\mathbf{V} = (\mathbf{I} - \gamma\mathcal{P})^{-1}\mathbf{R}}$$

Step 2: Form the Coefficient Matrix

Compute $\mathbf{I} - \gamma\mathcal{P}$ for $\gamma = 0.9$:

$$\mathbf{I} - 0.9\mathcal{P} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - 0.9 \begin{pmatrix} 0.7 & 0.1 & 0.2 \\ 0.1 & 0.7 & 0.2 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{pmatrix} = \begin{pmatrix} 0.37 & -0.09 & -0.18 \\ -0.09 & 0.37 & -0.18 \\ -0.3 & -0.3 & 0.7 \end{pmatrix}$$

Step 3: Write the Bellman Equations Explicitly

Expanding $(\mathbf{I} - \gamma\mathcal{P})\mathbf{V} = \mathbf{R}$:

$$\begin{aligned} 0.37\,V(O) - 0.09\,V(U) - 0.18\,V(A) &= -10 \\ -0.09\,V(O) + 0.37\,V(U) - 0.18\,V(A) &= +10 \\ -0.3\,V(O) - 0.3\,V(U) + 0.7\,V(A) &= -1 \end{aligned}$$

Solving this 3×3 linear system (via Gaussian elimination, Cramer's rule, or `np.linalg.solve`):

Results

For $\gamma = 0.9$:

$$\boxed{V(O) \approx -23.75, \qquad V(U) \approx +32.01, \qquad V(A) \approx +1.04}$$

Interpretation:

- $V(O) \approx -23.75$: The system tends to stay in O (70% self-loop), accumulating -10 rewards over time.
- $V(U) \approx +32.01$: The system tends to stay in U (70% self-loop), accumulating $+10$ rewards.
- $V(A) \approx +1.04$: Despite the -1 immediate reward, the $\frac{1}{3}$ chance of reaching U makes this state slightly positive.

Verification Across Discount Factors

γ	$\mathbf{V}(\mathbf{O})$	$\mathbf{V}(\mathbf{U})$	$\mathbf{V}(\mathbf{A})$
0	-10.00	$+10.00$	-1.00
0.1	-10.63	$+10.63$	-1.00
0.9	-23.75	$+32.01$	$+1.04$
0.999	-163.30	$+343.37$	$+88.03$

At $\gamma = 0$, only immediate rewards matter so $V(s) = R(s)$. As $\gamma \to 1$, future rewards accumulate and the values diverge significantly. Note how $V(A)$ flips from negative to positive around $\gamma \approx 0.5$ as the future value of potentially reaching U outweighs the -1 penalty.

Implementation

```
import numpy as np

def mrp(state_trans_probs, state_rewards, gamma):
    I = np.eye(state_trans_probs.shape[0])
    return np.linalg.solve(I - gamma * state_trans_probs, state_rewards)
```

Problem 2.1: Dynamic Programming (40 pts)

Task: Implement the `dynamic_programming` function. Use any tabular dynamic programming algorithm to exactly compute the optimal state-value function and an optimal policy for the AI Farm environment.

Algorithm: Value Iteration

Value iteration repeatedly applies the Bellman optimality update until convergence:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a) \cdot V_k(s') \right]$$

The optimal policy is extracted after convergence:

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}(s)} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a) \cdot V^*(s') \right]$$

Step 1: Initialize

Set $V(s) = 0$ for all states s. Choose convergence threshold $\theta = 10^{-8}$.

Step 2: Value Iteration Loop

For each state s and each action a:

1. Call `env.state_action_dynamics(s, a)` to get $r(s,a)$, next states $\{s'\}$, and probabilities $\{P(s'|s,a)\}$.
2. Compute $Q(s,a) = r(s,a) + \gamma \sum_{s'} P(s'|s,a) \cdot V(s')$.
3. Update $V(s) = \max_a Q(s,a)$.

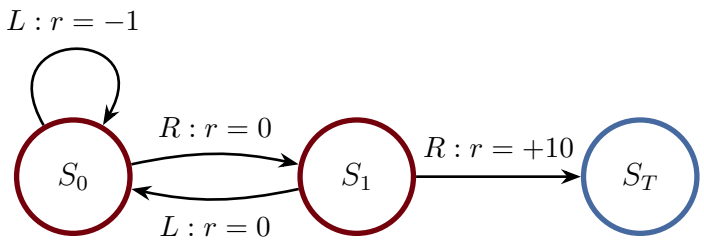
Repeat until $\max_s |V_{\text{new}}(s) - V_{\text{old}}(s)| < \theta$.

Step 3: Extract Policy

For each state s, compute $Q(s,a)$ for all actions and assign probability 1 to $\arg\max_a Q(s,a)$.

Worked Example: 2-State MDP

Consider a simple MDP with states S_0, S_1 and terminal state S_T, $\gamma = 1$:



Value Iteration (hand trace):

Iteration 1 (from $V_0(S_0) = 0,\ V_0(S_1) = 0$):

$$V_1(S_0) = \max\big(\underbrace{-1 + 1 \cdot V_0(S_0)}_{L{:}-1+0=-1},\ \underbrace{0 + 1 \cdot V_0(S_1)}_{R{:}0+0=0}\big) = 0 \qquad \pi(S_0) = R$$

$$V_1(S_1) = \max\big(\underbrace{0 + 1 \cdot V_0(S_0)}_{L{:}0+0=0},\ \underbrace{10 + 1 \cdot V_0(S_T)}_{R{:}10+0=10}\big) = 10 \qquad \pi(S_1) = R$$

Iteration 2 (from $V_1(S_0) = 0,\ V_1(S_1) = 10$):

$$V_2(S_0) = \max\big(\underbrace{-1 + 0}_{L{:}-1},\ \underbrace{0 + 10}_{R{:}10}\big) = 10 \qquad \pi(S_0) = R$$

$$V_2(S_1) = \max\big(\underbrace{0 + 0}_{L{:}0},\ \underbrace{10 + 0}_{R{:}10}\big) = 10 \qquad \pi(S_1) = R$$

Iteration 3: $V_3(S_0) = \max(-1, 10) = 10$, $V_3(S_1) = \max(10, 10) = 10$. **Converged.**

Iter	$\mathbf{V(S_0)}$	$\mathbf{V(S_1)}$	$\boldsymbol{\pi(S_0)}$	$\boldsymbol{\pi(S_1)}$
0	0	0	—	—
1	0	10	R	R
2	10	10	R	R
3	10	10	R	R

Results

$V^*(S_0) = 10$, $V^*(S_1) = 10$. Optimal policy: $S_0 \xrightarrow{R} S_1 \xrightarrow{R} S_T$, collecting +10 total reward.

Implementation

```
def dynamic_programming(env, states, state_values, gamma):
    theta = 1e-8
    while True:
        delta = 0
        for s in states:
            actions = env.get_actions(s)
            if not actions:
                continue
            v_old = state_values[s]
            q_vals = []
            for a in actions:
                r, nxts, probs = env.state_action_dynamics(s, a)
                q = r + gamma * sum(
                    p * state_values[ns]
                    for ns, p in zip(nxts, probs))
                q_vals.append(q)
            state_values[s] = max(q_vals)
            delta = max(delta, abs(v_old - state_values[s]))
        if delta < theta:
            break

    # Extract greedy policy
    policy = {}
    for s in states:
        actions = env.get_actions(s)
        probs = [0.0] * len(actions)
        if actions:
            best_q, best_i = float('-inf'), 0
            for i, a in enumerate(actions):
                r, nxts, ps = env.state_action_dynamics(s, a)
                q = r + gamma * sum(
                    p * state_values[ns]
                    for ns, p in zip(nxts, ps))
                if q > best_q:
                    best_q, best_i = q, i
            probs[best_i] = 1.0
        policy[s] = probs

    return state_values, policy
```

Problem 2.2: Model-Free Reinforcement Learning (40 pts)

Task: Implement the `model_free` function. Use a tabular model-free RL algorithm to

approximate the optimal action-value function. **Cannot** use `env.state_action_dynamics`.

Algorithm: Q-Learning

Q-learning is an off-policy TD control algorithm. The update rule is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

with ε-greedy exploration:

$$a = \begin{cases} \arg\max_a Q(s,a) & \text{with probability } 1-\varepsilon \\ \text{random action} & \text{with probability } \varepsilon \end{cases}$$

Step 1: Initialize

Set $Q(s,a) = 0$ for all state-action pairs. Choose hyperparameters:

- Learning rate $\alpha \in (0,1]$ (e.g., $\alpha = 0.1$)
- Exploration rate ε, decaying from 1.0 to 0.01
- Number of episodes N (e.g., $N = 50000$)

Step 2: Episode Loop

For each episode:

1. Sample start state $s \leftarrow$ `env.sample_start_state()`.
2. While s is not terminal:
   (a) Select action a via ε-greedy w.r.t. $Q(s,\cdot)$.
   (b) Take action: $(s', r) \leftarrow$ `env.sample_transition(s, a)`.
   (c) Update: $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$.
   (d) $s \leftarrow s'$.
3. Decay ε: $\varepsilon \leftarrow \max(\varepsilon_{\min},\ \varepsilon \cdot \varepsilon_{\text{decay}})$.

Worked Example: 2-State MDP with Q-Learning

Same 2-state MDP as Problem 2.1. Parameters: $\alpha = 0.1$, $\gamma = 1.0$, $\varepsilon = 0.3$.
Initialize: $Q(S_0, L) = Q(S_0, R) = Q(S_1, L) = Q(S_1, R) = 0$.

Episode 1

Step	s	a	s′	r	Q-update
1	S_0	R	S_1	0	$Q(S_0,R) = 0 + 0.1[0 + 0 - 0] = 0$
2	S_1	R	S_T	10	$Q(S_1,R) = 0 + 0.1[10 + 0 - 0] = 1.0$

Episode 2

Step	s	a	s′	r	Q-update
1	S_0	R	S_1	0	$Q(S_0, R) = 0 + 0.1[0 + 1.0 - 0] = 0.1$
2	S_1	R	S_T	10	$Q(S_1, R) = 1.0 + 0.1[10 + 0 - 1.0] = 1.9$

Episode 3

Step	s	a	s′	r	Q-update
1	S_0	R	S_1	0	$Q(S_0, R) = 0.1 + 0.1[0 + 1.9 - 0.1] = 0.28$
2	S_1	R	S_T	10	$Q(S_1, R) = 1.9 + 0.1[10 + 0 - 1.9] = 2.71$

After 3 episodes the Q-values are already trending toward the true values. The pattern is clear: each episode, $Q(S_1, R)$ absorbs more of the $+10$ terminal reward, and $Q(S_0, R)$ bootstraps off the improving $Q(S_1, R)$.

Results

After many episodes: $Q(S_0, R) \to 10$, $Q(S_1, R) \to 10$, matching the DP solution. The greedy policy $\pi(s) = \arg\max_a Q(s, a)$ yields $S_0 \to R$, $S_1 \to R$.

Implementation

```
import random

def model_free(env, action_values, gamma):
    alpha = 0.1
    eps = 1.0
    eps_decay = 0.9995
    eps_min = 0.01
    num_episodes = 50000

    for _ in range(num_episodes):
        state = env.sample_start_state()
        while True:
            actions = env.get_actions(state)
            if not actions:
                break
            # Epsilon-greedy action selection
            if random.random() < eps:
                idx = random.randrange(len(actions))
            else:
                idx = max(range(len(actions)),
                    key=lambda i: action_values[state][i])
            action = actions[idx]
            nxt, reward = env.sample_transition(state, action)

            # Q-learning update
            nxt_acts = env.get_actions(nxt)
            max_q = (max(action_values[nxt])
                     if nxt_acts else 0.0)
            td = reward + gamma * max_q
            action_values[state][idx] += (
                alpha * (td - action_values[state][idx]))

            state = nxt
        eps = max(eps_min, eps * eps_decay)

    return action_values
```